

# Interactive Data Analyst Core

## System Design & Architecture Report

---

### 1. Execution Result Design

---

#### How Execution Results are Represented

The system relies on a LangGraph state machine where the `execution_result` artifact bridges the Code Generation/Sandbox nodes and the Visualization/Generation nodes. This artifact is strictly represented as a **JSON-safe Python Dictionary**. If the executed Pandas code yields a DataFrame or Series, it is converted to a list of record dictionaries using `to_json(orient="records")` and re-loaded via `json.loads()`. A custom sanitization function (`_make_json_safe`) recursively scrubs any residual objects.

#### Why We Chose This Representation

This strict serialization design was necessitated by FastAPI and Pydantic limitations. Pandas is built on top of NumPy, meaning DataFrames naturally contain C-based data types like `numpy.ndarray` or `numpy.int64`. When FastAPI attempts to transmit the LangGraph state to the React frontend, Pydantic's JSON serializer violently rejects NumPy types, resulting in a 500 Internal Server Error. By forcing the execution result into purely native Python primitive structures (lists, dicts, strings, ints), we guarantee that the data can safely cross the API boundary to the web client.

#### Trade-offs Observed

The primary trade-off of this approach is **computational overhead and loss of object methods**. Because the state holds pure dictionaries instead of native Pandas DataFrames, downstream nodes (like the Visualization Agent) must constantly reconstruct the DataFrame (e.g., `pd.DataFrame(result)`) to perform their own operations. While this ensures web-safety, it introduces minor serialization/deserialization latency and requires nodes to safely handle varying input shapes (lists vs. scalar dictionaries).

## 2. Visualization Agent Interface

---

### Input State

The visualization agent triggers via the `visualization_node`. Its input consists of the `current_query` (to understand user intent) and the `execution_result`. A classifier ensures that the visualization node is only triggered if the execution was successful (i.e., no errors, no empty results, and not a scalar value).

### Output State

The output is stored in the `visualization_figure` state key. It contains a fully sanitized **Plotly JSON Dictionary** (generated via `json.loads(fig.to_json())`) or `None` if no visualization was deemed appropriate.

### Information Required for Visualization

To successfully generate a chart, the agent requires:

- The raw data payload (converted back into a pandas DataFrame).
- The schema/column names of that specific data payload.
- The user's explicit or implicit graphing instructions (e.g., "scatter plot colored by Major").

### Frontend Usage

The generated Plotly JSON is beamed directly via the FastAPI `ChatResponse` model to the React frontend. Because the Plotly library uses standardized JSON schemas, the React app simply passes this dictionary into a `<Plotly />` component, natively rendering interactive, hoverable, zoomable charts directly in the chat UI without generating static image files.

## 3. System Usage & Summary

---

Below is a log of system interactions demonstrating the agent's analytical capabilities across a dataset containing student majors, applications submitted, session durations, and galaxies explored.

**Q1: "What is the total number of applications submitted by each major?"**

**Generated Answer:** "Based on the data, Computer Science students submitted a total of 14 applications, Data Science students submitted 6, Design submitted 8, and Engineering submitted 2."

**Visualization:** A bar chart was generated showing the sum of applications on the Y-axis and the Majors on the X-axis.

**Q2 (Multi-turn Follow-up): "Is there a correlation between Session Duration and Applications Submitted? Visualize it as a scatter plot colored by Major."**

**Generated Answer:** "A scatter plot has been generated to show the relationship between Session Duration and Applications Submitted, categorized by major. Please refer to the chart below for the specific data points."

**Visualization:** An interactive Plotly scatter plot was successfully generated, utilizing the previously loaded dataset context without losing the thread of the conversation.

**Q3: "Which student explored the most galaxies?"**

**Generated Answer:** "The student who explored the most galaxies is S106, a Computer Science major, with a total of 8 galaxies explored."

**Visualization:** None (Scalar/Single row lookup).

**Q4: "What is the average session duration across all records?"**

**Generated Answer:** "The average session duration across all students is 76.84 minutes."

**Visualization:** None (Scalar calculation).

**Q5: "Filter the data to only show the raw records for Design students."**

**Generated Answer:** "Here are the records for the Design students. The data includes their session durations, applications submitted, and galaxies explored. For example, student S105 submitted 4 applications with a session duration of 145.5 minutes."

**Visualization:** A data table displaying the filtered list.

**Summary of Insights:** The system demonstrates high reliability in parsing natural language into exact pandas aggregations and filters. By separating the Code Generation agent from the Visualization agent, the system maintains strict boundaries, ensuring

data is accurately parsed before visual rendering is attempted. The multi-turn memory allows the user to iteratively explore data without repeating context.

## 4. Failure Case & Limitations

---

### The NumPy/Pydantic Serialization Crash

**What went wrong:** During testing, when the user asked to visualize data as a scatter plot, the code executed successfully in the sandbox and generated the Plotly figure. However, the system threw a `500 Internal Server Error` just as the response was being sent to the frontend. The terminal reported: `Unable to serialize unknown type: numpy.ndarray`.

**Why it happened:** Under the hood, Plotly utilizes `numpy.ndarray` objects to store graphing coordinates. When the visualization node called `fig.to_dict()` to extract the chart data, it unknowingly passed these raw C-based NumPy arrays into the FastAPI/Pydantic response model. Pydantic only understands native Python JSON types and immediately crashed upon encountering the foreign array structures.

**How the system was improved:** This critical limitation was resolved by implementing a rigorous sanitization step. Instead of extracting raw dictionaries, the visualization node now calls `json.loads(fig.to_json())`. Plotly's native `to_json()` method automatically scrubs all NumPy arrays into web-safe JSON formats, entirely eliminating the Pydantic serialization crashes and stabilizing the system architecture.